0.1 PART 0 · INTRODUCTION

Two

Robots have welded car bodies more precisely than humans for decades, yet they learned to fold a T-shirt only recently. Rules written by an engineer stop working in the open world, while a neural network trained on examples needs lots of data and can fail confidently without giving any signal. In this lesson you will compare both approaches on the same task, find where each one breaks, and see why the rest of the course follows the data path.

≈ 40 minutes | 2 labs and 5 interactives

PARADIGM 1

Write the rules

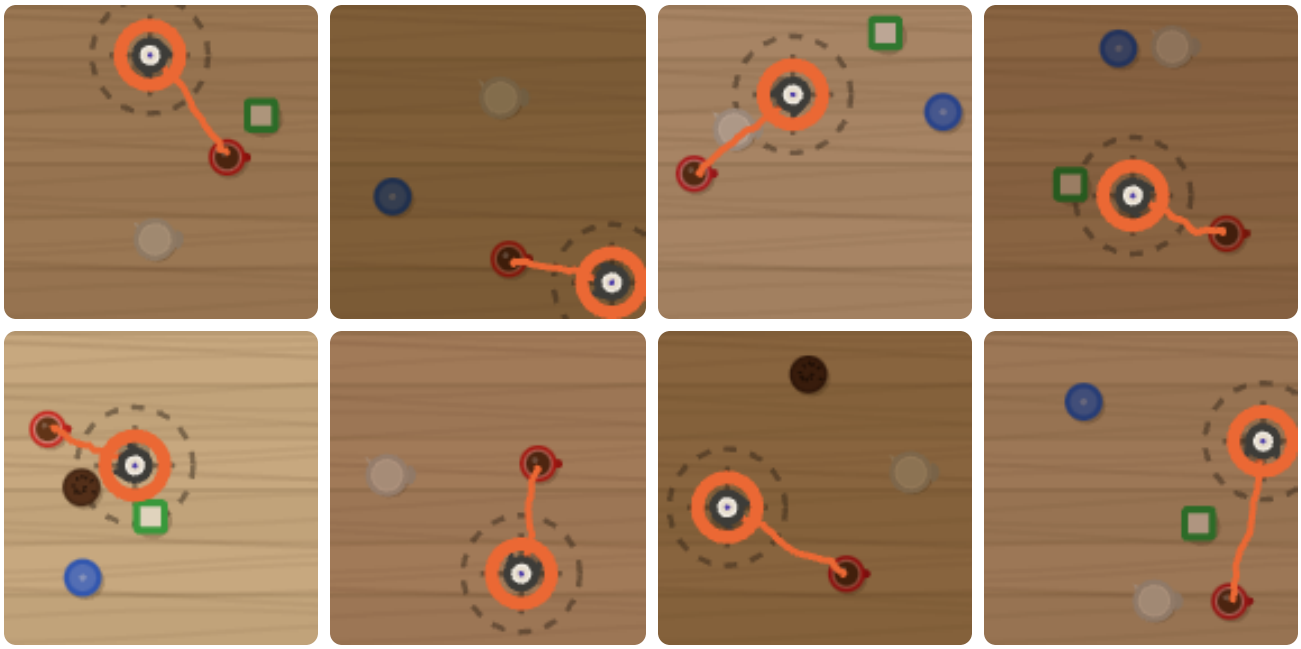
An engineer splits the task into modules and writes clear code for each. For the cup task it might look like this:

```
# perception: the cup is red, look for red pixels
# r, g, b — pixel color: red, green, blue
if r > 0.60 and g < 0.35 and b < 0.35:
    cup = centroid(pixels)  # center of the blob
# planning: a path to the cup around obstacles
path = astar(hand, cup, obstacles)
# control: hand velocity toward the next waypoint
v = pid(path.next - hand)
```

PARADIGM 2

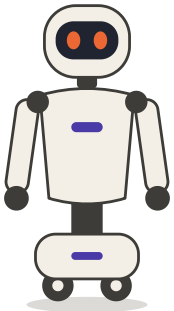
Show examples

A human guides the robot's hand to the cup many times with a remote controller, and a neural network learns to reproduce the motion. That human is called the operator, and each recording is a demonstration. Here are eight demonstrations from this lesson, a top-down view of the table:



robot's hand at the start — path the operator guided it along — cup

The rest on the table: a sugar bowl, a napkin holder, a milk jug and a bowl of coffee beans. From one recording to the next the objects and the cup stand in different places, and the kitchen light changes.



Ada*

Ada is a home humanoid on a wheeled base: a torso, a head and two arms like a human's, and a wheeled platform instead of legs. She is shared by all the labs in the course. Her environment is a simplified apartment: a kitchen, a hallway, two bedrooms. In every lesson Ada solves one task that real home robots face too: bring the hand to a cup, drive down the hallway to the kitchen, figure out which room she is in. On the same task we compare different methods, classical and learned.

The model is simplified: the scenes are two-dimensional, and the physics and sensors are reduced to what matters for the lesson's

topic. But the principles are the same as in real robots: sensors are noisy, motors have torque and speed limits, and the neural networks in the labs are real and train right in your browser.

* Named after Ada Lovelace, who wrote the first program for a computing machine.

DIAGRAM
Four control architectures: rules, a single neural network from camera to motors (end-to-end), a hybrid, and a two-level vision-language-action model

LAB 1
Three versions of Ada in one kitchen: find where each one breaks

LAB 2
Fix the failures with rules and with data, and compare what each fix costs

TASKS
Decide what in Ada's home to trust to a neural network and what to rules

SPECTRUM
How the approaches combine in real systems of 2026

Start the lesson ↓

1 PARADOX

Welding a car body is easier than folding a T-shirt

In 1961 **Unimate**, the first industrial robot, started work at a General Motors plant. Today manipulators weld car bodies to a fraction of a millimeter, around the clock and almost without defects. Yet robots learned to fold a T-shirt reliably only recently, although any teenager can do it.

Hans Moravec noticed why back in 1988: what comes to people without effort — seeing, grasping, walking — turns out to be the hardest for a machine. Chess and calculation are easy

for a computer, but the perception and motor skills of a one-year-old are not. This is called **Moravec's paradox**.

A factory robot is programmed: its motions are set in advance, and the parts always lie in the same places. The world is arranged around the robot, so every situation can be described by rules beforehand. At home you cannot arrange the world around the robot: fabric falls differently every time, and you cannot describe every case in advance.

That is why, instead of rules, a robot is increasingly given examples, and a neural network is trained on them. A model that takes a camera image and a command in words and directly outputs robot actions is called a **VLA** (vision-language-action). The term was introduced by the authors of RT-2 (Google DeepMind, 2023; lesson 3.3). One such model is **π0** (Physical Intelligence, 2024; lesson 3.6): it takes laundry out of a dryer and folds it. The manipulators it controls differ little from manipulators of ten years ago; what changed is the control system.



2016 End-to-end

Sergey Levine and colleagues trained a single neural network to output motor commands straight from a camera image, with no hand-written modules. This approach is called end-to-end. The network in Lab 1 is modeled on this work.

So a robot's behavior can be obtained in two ways: an engineer writes rules, or a neural network learns from examples. How do the control systems built these two ways differ?

2 ARCHITECTURES

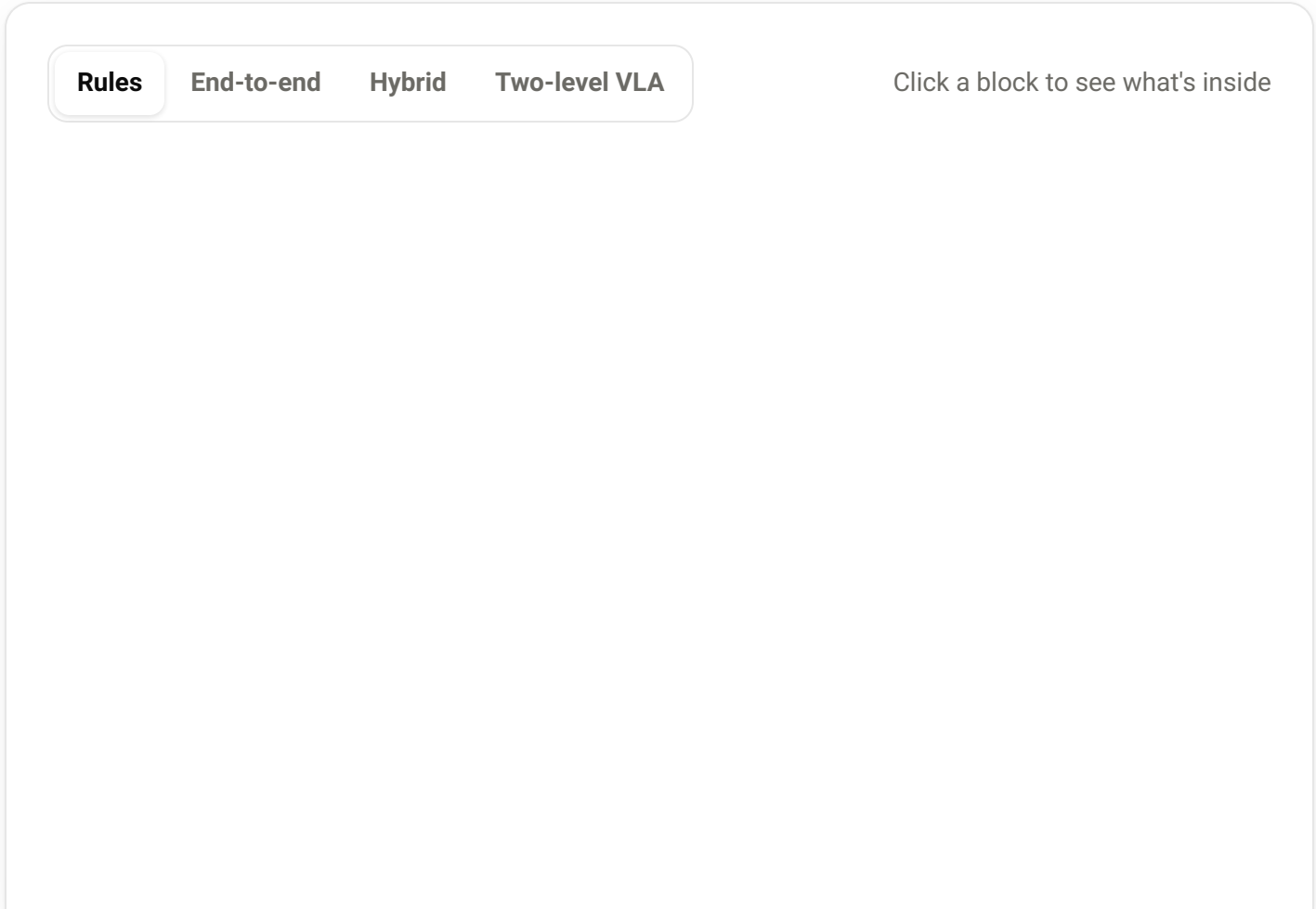
How a robot's control system is built

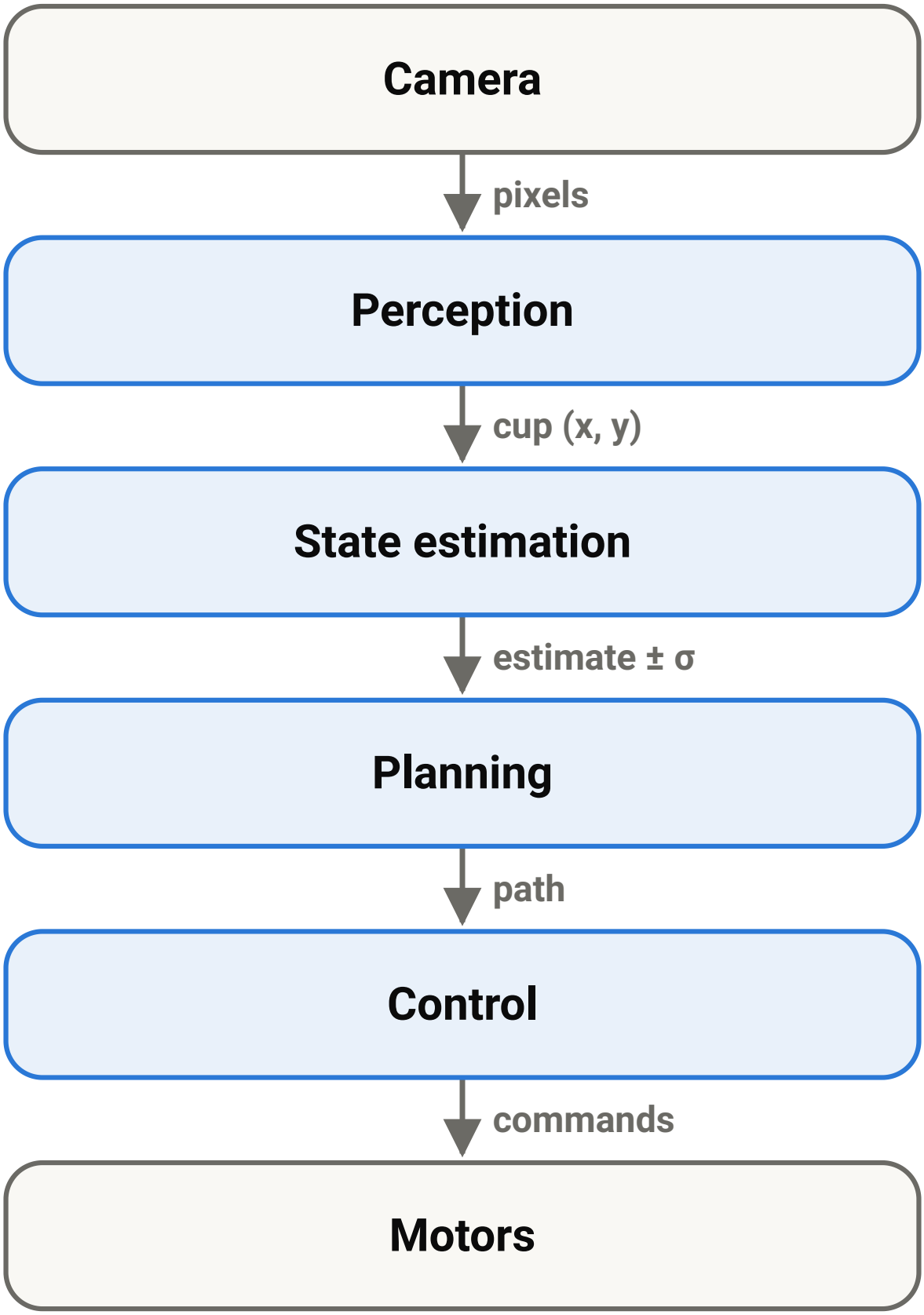
The difference is easiest to see on one task. Take the one that runs through the whole lesson: Ada stands at the kitchen table, a camera looks at the table from above, and the hand has to

reach the cup.

The classical stack splits the task into modules, which is why it is also called modular: perception finds the cup, state estimation refines where it is, the planner builds a path, the controller drives the hand. Every module runs on rules and heuristics written by an engineer. In the diagram and the labs of this lesson this approach is called "Rules". The data approach replaces the whole chain with a single neural network: the operator guides the hand to the cup many times, and the network learns to turn camera pixels directly into motor commands. This is end-to-end. A network that picks the robot's next action from what the robot sees is called a **policy**. A rules-based program can be a policy too, but the word is more often used for a neural network. Between rules and end-to-end there are hybrids.

The VLAs from the previous section are end-to-end too: an image and a command go in, actions come out. In RT-2 and π0 this is a single model. But a large model is hard to run often, while the arms have to be controlled fast. So Helix from Figure and GR00T N1 from NVIDIA (both from 2025, lesson 3.7) split the VLA into two levels: a slow vision-language model understands the scene and the command, and a fast policy drives the arms. This two-level architecture is one way to build a VLA. Switch architectures and click the blocks.





Each module solves its own task and passes the result to the next one in a strictly agreed format.

Click any block in the diagram

The classical, or modular, stack: five modules on rules and heuristics and four interfaces between them. Each module can be tested separately — and each has to be designed by hand. The σ on the arrow is the estimation error: how unsure the module is about where the cup is.

Rules : strengths and weaknesses

+ **Guarantees.** The planner avoids a detected obstacle by construction; the controller's stability is proven mathematically.
+ **Rate and precision.** Control loops run at hundreds and thousands of hertz.
+ **Transparency.** Each module is tested separately; a failure shows up at a specific interface.
− **Open world.** The rule "red means cup" lives as long as cups are red and the light is daylight. Every new situation needs a new rule, and there are infinitely many situations. This is called the *long tail*.

Learning from data : strengths and weaknesses

+ **Generalization.** The network finds features on its own that an engineer would never think to program.
+ **No hand-made interfaces.** The whole system is optimized for the final goal; no information is lost between modules. In the lesson's reference paper (Levine et al., 2016) the robot inserted a block into a shape sorter in 96% of attempts, while a modular version with separate pose estimation never did: a centimeter of error at the interface killed a task with millimeter tolerance.
− **Needs lots of data.** Hundreds of examples even for a simple task, thousands of hours for a general-purpose model.
− **No guarantees and silent failures.** Outside its data the network behaves unpredictably and does not report an error.

A similar shift has already happened with text: language processing moved from rules to large language models. But language models trained on text from the internet, and there is no such reserve of data for robots. How this limit is worked around is covered in Part 4.

On paper, rules give guarantees and transparency but cannot cope with the open world. Data gives generalization but needs lots of examples and can fail without giving any signal. Where does each approach actually break?

🔬 Lab 1

Three versions of Ada, one task

Where does each approach actually break? Lists of pros and cons won't tell you: you need to put the approaches side by side and change the conditions one at a time.

Three versions of Ada get identical arms and the same kitchen. In the rules version every module is written by an engineer. In end-to-end a single neural network runs from camera to motors. In the hybrid a neural network only finds the cup, rules compute the path and the motion, and a safety rule stops the hand near an obstacle. The networks are training right now in your browser on 200 demonstrations: these are real networks, not animation. In five experiments the conditions change one at a time: the light, an obstacle in the way, the color of the cup. Before each one predict who will reach the cup, then launch all three with the button under the scenes. After five experiments free mode opens: there you can move objects, change the light, the cup and the camera noise.

1 An ordinary day | 2 Evening | 3 Kettle in the way | 4 Evening and kettle | 5 Teal cup | 🔒 Free mode

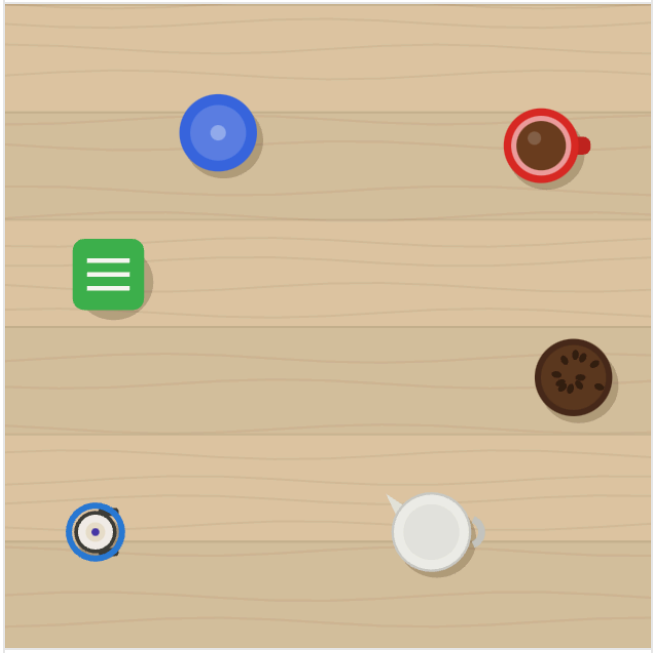
Experiment 1 of 5 · An ordinary day

Let's start with a situation all three are ready for: daylight, a red cup, nothing in the way. The engineer wrote the rules for it; the operator recorded the demonstrations in it.

Your prediction: who will reach the cup? ○ Rules ○ End-to-end ○ Hybrid ○ Nobody

Rules

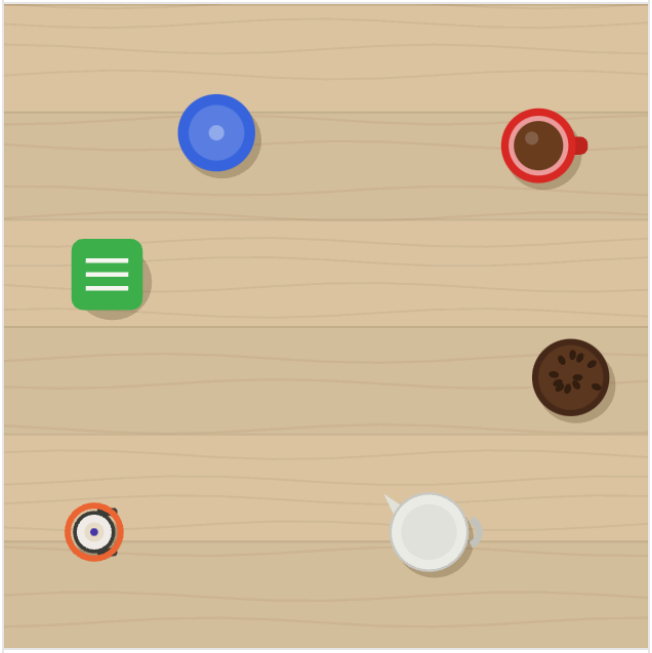
color rule finds the cup → filter refines the position → A* builds a path → controller drives the hand



Ready to run

End-to-end

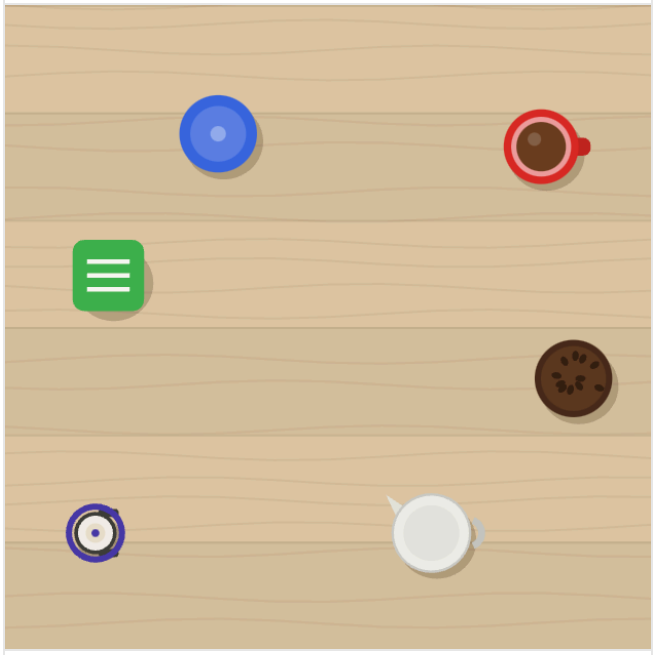
pixels + hand position → neural network → motor command



Ready to run

Hybrid

neural network finds the cup → filter → A* → controller; a safety rule stops the hand near the danger zone



Ready to run

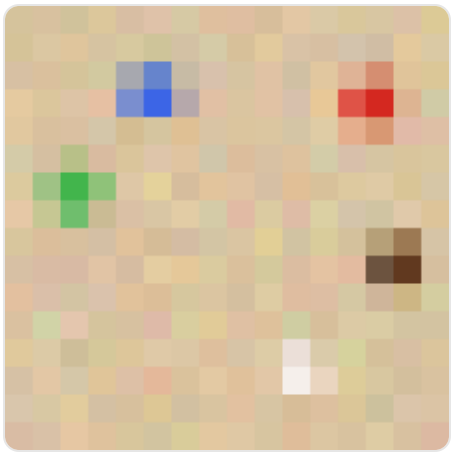
Top-down view of the table. The three versions of Ada share the same scene; only the control system differs. What each icon means is in the legend below.

▸ **Legend: icons on the scene**

▶ Run | ⏸ Pause | ↺ Reset

Make your prediction above and run: all three versions of Ada will move at the same time. The debrief will appear right here.

What the camera sees



Pixels | Rule

What each architecture sees

To Ada the table is 256 colored pixels. The rules version looks at them through rules an engineer wrote. The neural network learned where to look on its own: below are its attention maps. The darker the cell, the more weight that pixel has when computing a "point of interest".

end-to-end · p1

end-to-end · p2

end-to-end · p3

end-to-end · p4

hybrid · "cup"

Loud and silent failures: statistics instead of one scene

One scene can mislead. Run 40 random scenes under the current conditions and see how each version fails.

📊 **Run 40 scenes**

In the reliability literature these failures are called signaled and unsignaled. For brevity we say "loud" and "silent".

Each approach failed in its own scenario. The rules did not find the cup in the evening and did not recognize the teal one, but both times they stopped and reported the failure. The end-to-end network crashed into the kettle and confidently drove to the bowl instead of the teal cup. The hybrid handled the evening and the kettle, but also mistook the teal cup for the bowl. Can these failures be fixed, and what will it cost each approach?

🔬 Lab 2

Fixing failures: rules versus data

How do you fix the failures from Lab 1? The approaches have different tools. Rules are fixed with rules: the engineer sees the failure and adds code. A neural network is fixed with data: the operator records new demonstrations, and the network is retrained.

Walk both paths on the same Ada and compare what each one fixes and what it costs. In the "Rules path" card add fixes; in the "Data path" card change the number and variety of demonstrations and retrain the networks. The changes apply immediately to Ada in Lab 1.

Rules path

Perception rules the engineer wrote in daylight. Add fixes for the failures from Lab 1.

```
def is_cup(r, g, b):
    return r > 0.60 and g < 0.35 and b < 0.35   # red

def is_obstacle(r, g, b):   # dark and gray — the kettle
    return mean(r, g, b) < 0.25 and spread(r, g, b) < 0.08
```

- [ ] **White balance by table color (fixes the evening)**
- [ ] **A rule for teal cups**
- [ ] **A rule for the next surprise…**

RULES WITH THESE FIXES, 40 SCENES

Normal scenes	✅ 100%
🌆 Evening light	❌ 0%
🫖 Kettle in the way	✅ 100%
💙 Teal cup	❌ 0%
🌆+🫖 Evening and kettle	❌ 0%

💡 Rules go around the kettle without a single demonstration: the obstacle is described by a rule, and the path avoids it. But every new kind of cup has to be explained to them with a new rule.

4
lines of rules

0 of ∞
cases fixed

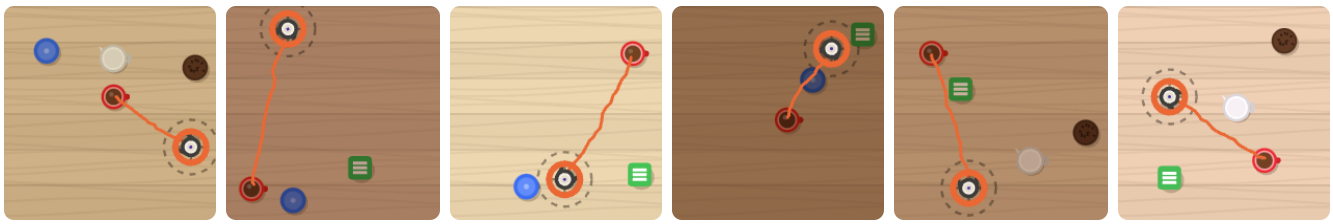
Data path

How many demonstrations did the operator record, and how varied are they? Both neural networks train on this data: the end-to-end policy and the hybrid's perception.

Demonstrations **200**

≈ 1 h 40 min of teleoperation, if one demonstration takes 30 seconds including setting up the scene

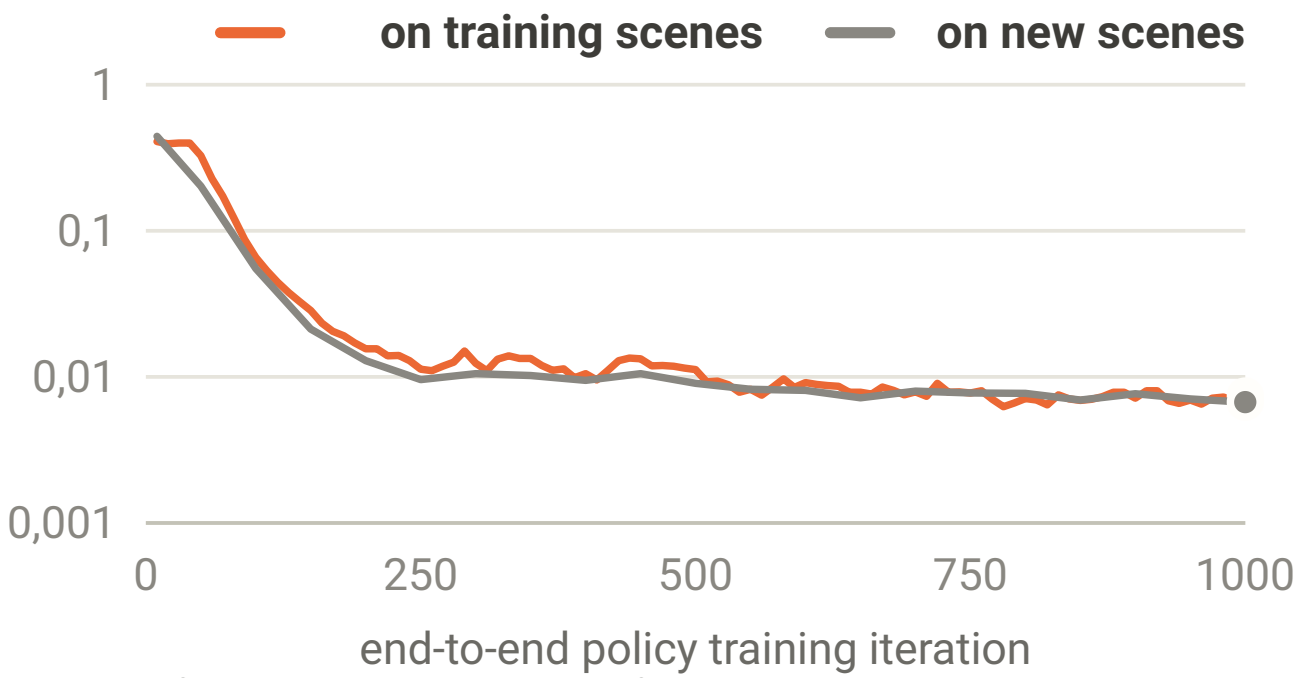
☐ **Varied lighting in the data: morning, day, evening**



hand at the start — path the operator guided it along — cup

🧠 Train the networks | 📈 **Build the full curve**

The default networks are trained on 200 demonstrations. Change the data and retrain.

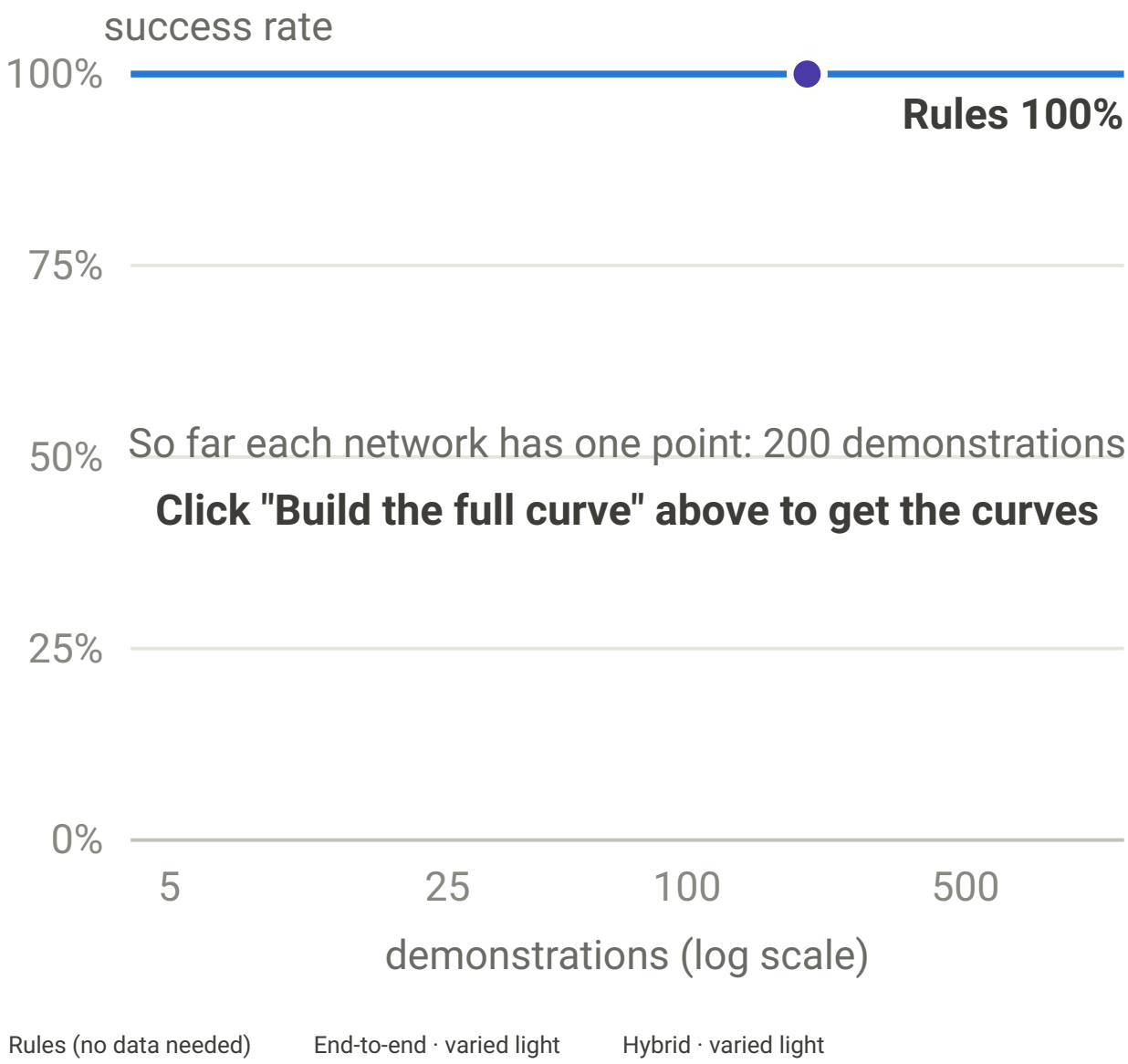


The loss is how far the network's commands diverge from the operator's commands in the demonstrations; the scale is logarithmic, each division is 10 times smaller than the previous one. The orange line is the loss on the scenes the network trains on, the gray one on new scenes it has never seen. If the gray line is noticeably above the orange one, the network memorized the examples but did not learn to generalize.

Success versus the number of demonstrations

Each point is one training run checked on 40 new scenes. Rules need no data: their level is a horizontal line. At first each network has one point: the 200 demonstrations it trained on when the lesson opened. The curves appear after the "Build the full curve" button above: the networks will train on 5, 10, 25, 50, 100, 200, 500 and 1000 demonstrations.

Normal | 🌆 Evening | 🫖 Kettle | 💙 Teal | 🌆+🫖 | **Table**



The latest model under all conditions

Share of the 40 scenes where Ada reached the cup. The networks are trained on 200 demonstrations with varied lighting.

Conditions	Rules	End-to-end	Hybrid
Normal scenes	✅ **100%**	✅ **100%**	✅ **100%**
🌆 Evening light	❌ **0%**	✅ **100%**	✅ **100%**
🫖 Kettle in the way	✅ **100%**	❌ **0%**	✅ **100%**
🩵 Teal cup	❌ **0%**	❌ **0%**	❌ **0%**
🌆+🫖 Evening and kettle	❌ **0%**	❌ **0%**	✅ **100%**

⚖️ **Both paths fix only what was anticipated in advance.** A rule is cheap but closes one case, and there are infinitely many cases. Worse, rules start interfering with each other: white balance fixes the evening but amplifies noise and breaks the kettle detector. Data is expensive: every demonstration is operator time. But it covers many variations at once that nobody listed — if those variations made it into the demonstrations.

Notice how little data the hybrid needs: its network learns only "where the cup is", not the whole behavior. Intermediate labeling saves data. Large models use the same idea differently: they first learn from huge general data, and then a few examples are enough for a new skill (Part 3).

Each path has its price. So which approach should you choose?

3 TRADE-OFFS

The price of each approach

To choose an approach, let's gather the observations from the two labs into one table. The last column says where you saw it.

Property	Rules	End-to-end	Hybrid	Where you saw it
Generalization to new conditions	each one needs a new rule	if the conditions are covered by data	thanks to neural perception	
Guarantees and safety	follow from the algorithm	none, only statistics	thanks to the classical modules	

Property	Rules	End-to-end	Hybrid	Where you saw it
Debuggability	you see which module failed	a "black box"	visible, but perception is opaque	
Nature of failures	mostly loud	silent and confident	silent if perception is wrong	
Data needs	almost none	hundreds of demonstrations	several times less than end-to-end	
Errors at module interfaces	accumulate: each module optimizes its own goal	no interfaces, everything learns for the goal	fewer interfaces	

🧭 **How the work is divided.** No approach wins at everything. Neural networks get what rules cannot describe: open-world perception, language, soft objects. Rules stay where guarantees, rate and precision are needed: speed limits, safety zones, emergency stop. How do you apply this to specific tasks?

⚠️ **Misconception: "end-to-end is always better, it's only a matter of data".** Guarantees do not come from data. A network that goes around the kettle 99 times out of 100 may drive into it on the hundredth, and nobody will know in advance.

⚠️ **Misconception: "a VLA controls the motors directly".** Vision-language-action models output goals a few dozen times per second, while the motors are driven by controllers at around a kilohertz. The boundary is shifting: in 2026 the bottom layer of some robots became neural too. Details in Lesson 0.3 and in Part 6.

🧩 Tasks

Who gets what in Ada's home

Let's test the rule "guarantees go to rules, the open world goes to neural networks" on specific tasks. Sort 12 tasks from Ada's home into three bins. For each one, ask yourself what matters more here: hard guarantees, or open-world perception and dexterity. Drag the cards, or click a card and then a bin.

🐈 **Don't run over the cat**

🧺 **Fold a towel**

🗣️ **Understand a request: "bring me something to drink, just not cold"**

🔌 **Drive to the charger on a known map**

🗄️ **Open an unfamiliar drawer**

🗓️ **Remember that we water the plants on Tuesdays**

🔍 **Find your own mug on the table among others**

🧒 **Slow down when a child is nearby**

🧸 **Pick up scattered toys**

🖐️ **Stop on the emergency button**

💧 **Pour water into a glass without spilling**

🥤 **Don't crush a plastic cup**

Rules

known environment, hard guarantees, ordinary code

End-to-end
a neural network learns from data: perception, language, the open world, soft and granular objects

Hybrid
a neural network understands the situation, a rule guarantees safety

Check | Start over | Show the experts' answers

4 REAL SYSTEMS

How the approaches combine in 2026

Real robots divide the work the same way: products built only on rules or only on data are rare. Here is the approximate position of well-known robots: the further right, the more of the behavior is learned. Click a dot.

← rules end-to-end →

Welding robot at a car plant

- da Vinci surgical robot
- Amazon warehouse robot (Kiva)
- Roomba j7 robot vacuum
- **Waymo robotaxis**
- Boston Dynamics Atlas with a Large Behavior Model
- Figure humanoid with the Helix model
- Tesla FSD v12 and later
- π0 and its successors from Physical Intelligence

Waymo robotaxis **hybrid**

In 2025 Waymo described a "think fast and think slow" design: a neural network fuses cameras, lidars and radars, a Gemini-based model reasons about the scene, and a separate verification layer watches over safety. The company explicitly contrasts it with pure end-to-end.

🔩 Even in the most end-to-end systems, classical controllers usually run at the bottom level. The network outputs target joint positions dozens of times per second, and the motor control loop tracks them at hundreds and thousands of hertz.

Real systems combine both approaches, but the boundary is shifting toward data. With each generation the learned part grows: first neural networks took over perception, then planning, now motion control too. Why does the industry bet on data, with all its weaknesses?

5 WHAT'S NEXT

Why the course follows the data path from here

The industry bets on data because rules do not scale. You cannot write a rule for every situation: a kitchen has infinitely many cups, lights and kinds of mess. This long tail is exactly what the rules version of Ada ran into: every fix closed one case and dragged in the next.

Data scales differently: to teach a network something new, you do not need to know in advance how the world works — you need to show enough examples. That is why the course follows the data path from here, and brings out the classics when there is no way around them.

The main limit became the data itself, and the industry is collecting it in volumes unheard of in robotics. π0 in 2024 was trained on 10,000+ hours of robot demonstrations. GEN-0 in 2025 — already on 270,000 hours, but these are recordings of people working with their hands, captured by wearable devices.

It is still a long way to the texts that language models train on: Ken Goldberg called this gap "the 100,000-year data gap". The estimate is rough, he himself calls it a thought experiment, but it conveys the order of magnitude correctly. How the gap is being closed — with teleoperation, human video, simulation and data from robots already at work — is covered in Part 4.

🧭 **What comes next.** The main path of the course starts with Part 1: imitation learning, large models, data, reinforcement learning, humanoids and world models. Before that come lessons 0.2, 0.3 and 0.4: the robot's body, motion control, state estimation and planning.

6 CHECK YOURSELF

Six common misconceptions

1. **How does the hybrid in the diagram differ from rules?**

- Perception is done by a neural network, and the other modules with their inputs and outputs are the same as in rules
- It is end-to-end with a safety rule added
- It has no modules; the network outputs motor commands directly

2. **The end-to-end network from Lab 1 sees a teal cup that was not in the data. What does Ada do?**

- Stops and reports that she does not recognize the object
- Drives to the cup: a cup is a cup
- Confidently drives to another object

3. **Is a two-level VLA like Figure Helix a hybrid of rules and a neural network?**

- Yes: the fast level is a PID controller
- No: both levels are learned; they are split by speed
- Yes: the slow level is a rules-based planner

4. **Why don't robots have their own "internet of data"?**

Robots are too expensive

Nobody creates action trajectories "just because": each one has to be obtained

Companies keep their data secret

5. **The hybrid from Lab 1 failed on the teal cup. What let it down?**

The safety rule triggered

The planner laid out the wrong path

Neural perception pointed at the wrong object, and the rules safely drove the hand to the wrong goal

6. **What does a robot's neural policy usually output?**

Targets for the joints or the gripper, from a few to a couple of hundred times per second, executed by controllers

A text plan that a program then reads

Motor torques a thousand times per second

7 TAKEAWAYS

Lesson takeaways

1 **Rules** are modules with clear interfaces: guarantees, rate, transparency. The weakness is the open world and the long tail of situations.

2. **Learning from data**: the network finds features on its own and generalizes. The weakness is that it needs lots of data, and failures can be silent: the network does not know it is wrong.

3. A network generalizes only to what the data covers. What matters is not just the number of examples but their **variety**.

4. Modern systems are **neural models** with a safety net of rules. The learned part grows with every generation.

5. **The main limit is data.** That is why the course follows the data path from here and looks at how data is obtained and how models learn from it.

Assignment

A one-page essay. Pick a real product: a humanoid, a warehouse robot or a robot vacuum. Break down its stack: which parts are learned, which are programmed, and why exactly that way. Lean on the trade-off table from this lesson.

Resources

π0: Our First Generalist Policy
Physical Intelligence, 2024 — video with laundry folding

Good old-fashioned engineering can close the 100,000-year “data gap” in robotics
K. Goldberg, Science Robotics, 2025

End-to-End Training of Deep Visuomotor Policies
S. Levine, C. Finn, T. Darrell, P. Abbeel, 2016 — the lesson's reference paper and the prototype of the network in Lab 1

Open X-Embodiment
An attempt to build a shared robot dataset: over a million trajectories, 22 robot types

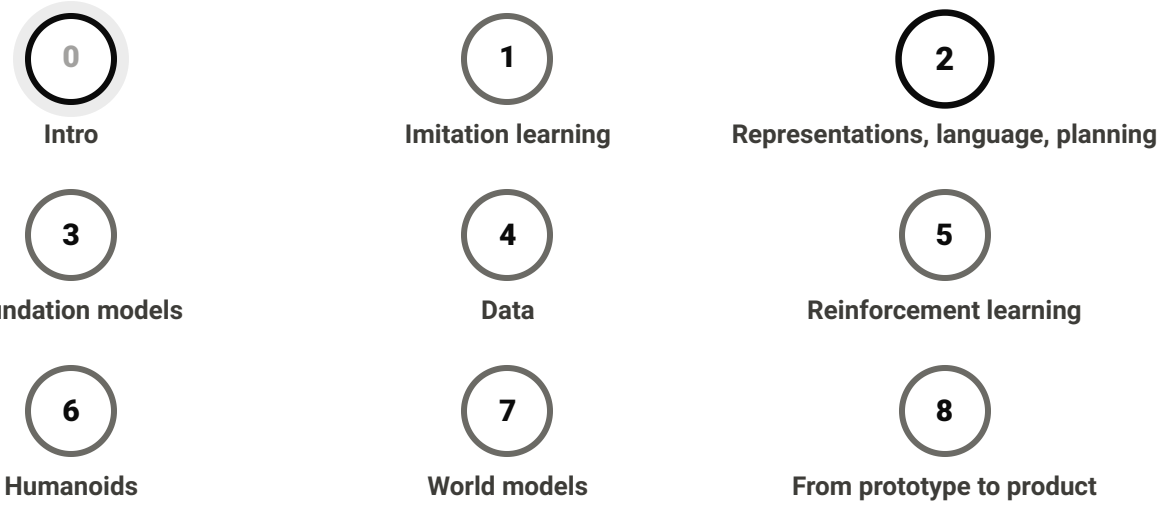
Robotic Manipulation

R. Tedrake, MIT — classics and learning in manipulation

➡️ **Next: how a robot is built.** In this lesson Ada had a perfect body: the network issued a command, and the gripper moved exactly where it was told. In a real robot the command is executed by motors with gearboxes and by controllers, and the robot learns about itself and the world through noisy sensors, each with its own rate. What a policy really receives and outputs, and what it controls — Lesson 0.2.

Course map

Nine parts. Click a station to see which question from this lesson is explored there.



Part 2 · Representations, language, planning

How to tie actions to scene geometry and words

Question from this lesson: How Ada can understand "hand me the blue mug" and find the right spot on the table.

Was the lesson clear?

All clear | Hard in places | Mostly unclear

Found a mistake or inaccuracy? Report it on GitHub

Found this lesson useful? Say thanks — that is how the author knows the course is being read.

Say thanks 88

Lesson 0.1 of the course "Modern Robotics and Physical AI". All simulations and neural networks run locally in your browser.

Course glossary — terms with definitions and links to the lessons.

Privacy policy